www.focus-microwaves.com

JANUARY 19, 2026

RAPIDMS API MANUAL

CREATED BY: LOKESH

MESURO LTD

PENCOED TECHNOLOGY PARK, PENCOED, CF35 5HZ, UK

INTRODUCTION

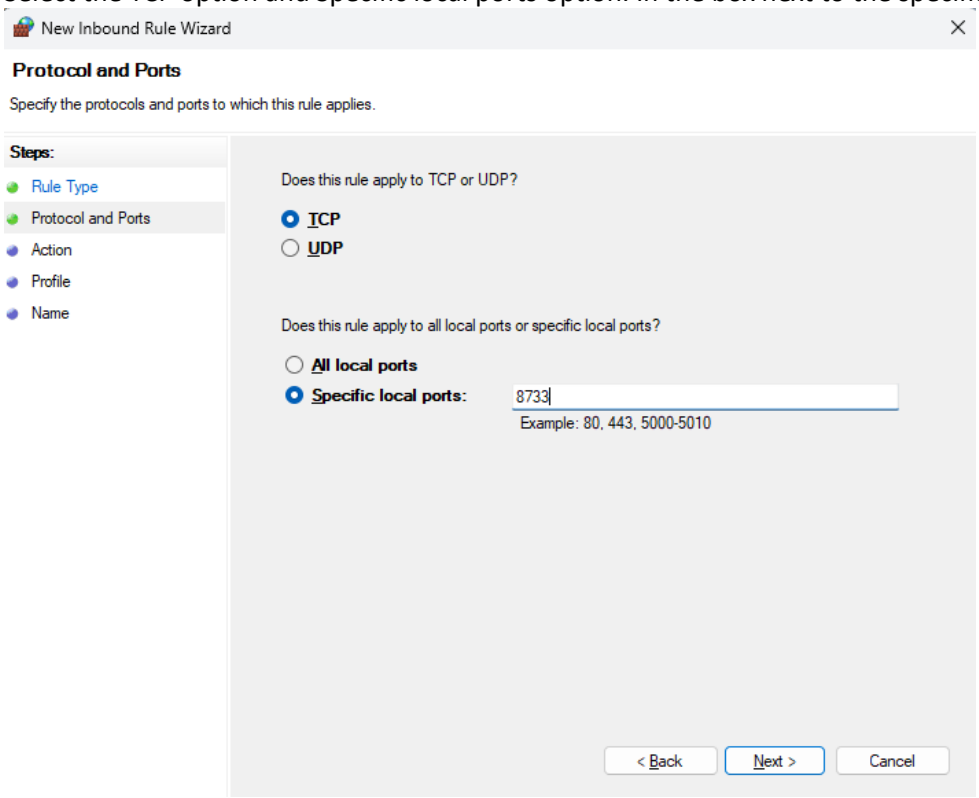
The document describes the RapidMS REST api interface exposed by the main software as well as a .NET client wrapper for the same. Most of the function definition and description are the same between the two. The api functionalities are shared between the RapidMS VT, DLP and DT systems and will support the features for which the RapidMS license exist. The DLP system does not support modulated measurements. In most cases, the user is recommended to use the .NET wrapper client api interface by importing the api dll into the client application.

Before describing the api interface, for the api interface to work correctly, the windows environment needs to be setup to accept and communicate with the client. Since the systems mentioned run on Windows platform, the document will talk about setup on a windows environment only. If your environment is something else, please contact Focus Microwaves support.

FIREWALL CONFIGURATION

The windows firewall needs to be configured in the measurement system as well as the client system. Please follow the steps below for the same.

- Open “Windows Defender Firewall with Advanced Security” from Windows start menu. (If you have a third party firewall, please contact the vendor and ask them for the equivalent instructions. Their support should be able to guide you on the equivalent steps below.)
- Select “Inbound Rules” on the left side of the application.
- Click on “New Rule…” on the right side.
- A popup wizard with settings to create a new inbound rule will open.
- Select the Port option and click Next.
- Select the TCP option and Specific local ports option. In the box next to the specific port, enter 8733.



- Click Next and select Allow the connection option.
- Click Next and select all 3 profiles.
- Click Next and enter a name for the rule. Provide a small description and click Finish.
- The new rule should be added to the Inbound rule list.

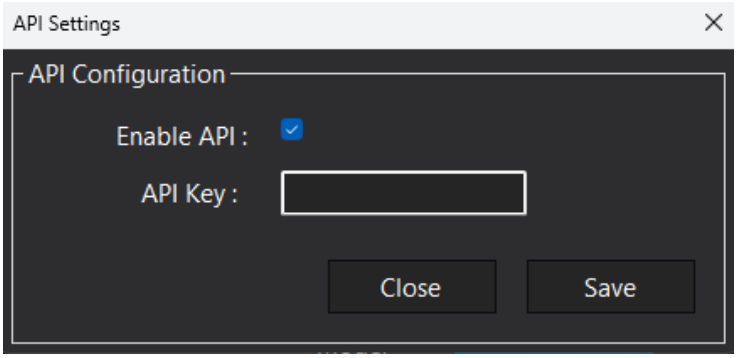
Once the inbound rule is added, select Outbound Rules from the left side of the application. Click on “New Rule…” from the right side and follow the same steps as the inbound rule creation. Please note to select “Allow the connection” option in the action tab when you do the configuration.

Based on the antivirus (including Microsoft defender), you may want to mark the RapidMS (and client) application as safe so as not to be interfered by the antivirus.

The same setup for Inbound and Outbound rule must be done in the client system as well. If you already have the RapidMS application running, then you should restart the application after the setup.

APPLICATION API CONFIGURATION

To use the RapidMS API, the API license option must be available. If the license option is available, then the API can be enabled in the software. To enable the API, select the Utilities menu option and then the API sub-menu option in the RapidMS software. A small pop up will appear with the option to enable the API as well as a box to enter the API key.



The api key is optional. If it is configured, then the remote user must provide the key when connecting to the system from the client system. The key is then authenticated against the one setup here.

IRapidMSClientAPI

The interface is a .NET wrapper for the RapidMS REST API. This interface wraps the RapidMS remote REST API interface providing an alternate way to interact with the RapidMS software. The implementation of the interface (RapidMSClientAPI class) forwards the function calls to the RapidMS REST API server, which then finally gets processed by the RapidMS software. Both the client API interface and class can be COM registered.

Public Member Functions

bool	**AbortMeasurement** ()
	The method aborts measurement if the system is running a sweep measurement.
bool	**Connect** (string remoteMachineAddress, int port, string key)
	The method lets remote user connect to the RapidMS system.
bool	**Disconnect** ()
	The method sends a disconnected request to RapidMS system.
bool	**DoMeasurement** (string filePath)
	The method requests the RapidMS system to perform measurement.
bool	**EnableTargetFromS1P** (bool enableS1pTarget)
	The method configures if the impedance targets should be set from s1p data.
string[]	**GetAvailableWaveforms** ()
	The method gets the list of modulated waveforms configured in the system.
double	**GetDCCurrent** (int dcSupply)
	The method gets the dc current measured on the specified dc supply.
bool	**GetDCOnOffStatus** (int dcSupply)
	The method requests the status of the dc supply.
double	**GetDCVoltage** (int dcSupply)
	The method gets the dc voltage measured on the specified dc supply.
string	**GetDeviceInfo** ()
	The method returns the RapidMS system information.
string	**GetInitParams** ()
	The method returns the initialization parameters.
bool	**GetMeasModeStatus** ()
	The method checks if the system is ready for measurement.

bool	**GetMeasuredABWave** (int harmonic, string selWaveform, out double[] waveform_real, out double[] waveform_imag)
	The method returns the last measured corrected A / B waves.
int[]	**GetPassiveTunerCal** (bool isLoad)
	The method gets the passive tuner calibrations in the passive tuner.
string	**GetReferenceLevels** ()
	The method retrieves the calibrated reference levels.
double[]	**GetSelectedFrequencies** ()
	The method returns the frequencies selected for measurement.
string	**GetSingleMeasuredData** ()
	The method returns the measurement result from single measurement.
bool	**GetSingleMeasuredData** (out string[] names, out double[] values)
	The method returns the measurement result from single measurement.
string	**GetStatus** ()
	The method requests the status of RapidMS system.
string	**GetSweepData** ()
	The method returns the measurement result from sweep measurement.
bool	**IsMeasurementCompleted** ()
	The method checks if the measurement has completed or not.
bool	**LoadInitializationFile** (string filePath)
	The method sets the initialization file to be loaded for impedance measurements.
bool	**LoadSourceCalibration** (string filePath)
	The method lets the user load a source calibration file.
bool	**LoadVectorCalibration** (string filePath)
	The method lets the user load a vector calibration file.
bool	**SetApullMode** (bool isApull)
	The method sets A-pull mode instead of Load-pull mode.
bool	**SetCenterFrequency** (double frequency)
	The method sets the center frequency for impedance measurement.
bool	**SetContinuousRf** (bool keepRfOn)
	The method sets the flag to keep rf on continuously.
bool	**SetDCBias** (int dcSupply, double setVoltage_V)
	The method sets the voltage of the specified dc supply.

`bool` **`SetIFBandwidth`** `(int targetBandwidthHz)`
The method sets the IF bandwidth of the system when in CW mode.
`bool` **`SetInitialization`** `(double initPowerdBm, string saveFilePath, bool skipGainCheck)`
The method sets the initialization settings.
`bool` **`SetLoadSourcepull`** `(int harmonic, bool enableLoadpull, bool enableSourcepull, int iterations, double accuracydB)`
The method sets impedance measurement settings for load-pull and source-pull.
`bool` **`SetLoadTargetPoint`** `(int harmonic, float targetReal, float targetImag)`
The method sets the load target point for impedance measurement.
`bool` **`SetLoadTargetPointPolar`** `(int harmonic, float targetMag, float targetPhaseDeg)`
The method sets the load target point for impedance measurement.
`bool` **`SetMeasurementMode`** `(int mode, int measType)`
The method sets the measurement mode and measurement category.
`bool` **`SetMeasurementType`** `(bool isSweep)`
The method sets the impedance measurement type.
`bool` **`SetModulatedBandwidth`** `(int targetBandwidth)`
The method sets the modulated bandwidth requested by the user.
`bool` **`SetModulatedWaveform`** `(string waveformName)`
The method sets the selected modulated waveform to be measured.
`bool` **`SetPassiveTunerCal`** `(bool isLoad, int tunerCalId)`
The method sets the selected passive tuner calibration.
`bool` **`SetPowerLevel`** `(double powerLeveldBm)`
The method sets the power level for measurement in dBm.
`bool` **`SetPowerSweep`** `(bool enable, double startdBm, double stopdBm, double stepdBm)`
The method enables and sets the power sweep settings.
`bool` **`SetPowerSweepCompression`** `(bool stopAtCompression, bool useMaxGain, double compressiondBm)`
The method sets the power sweep compression condition.
`bool` **`SetPowerSweepMaxAcprAdjCheck`** `(bool enable, double limit)`
The method sets power sweep max ACPR adjacent condition.
`bool` **`SetPowerSweepMaxDcInICheck`** `(bool enable, double limit)`
The method sets power sweep max input dc current condition.

bool	**SetPowerSweepMaxDcOutICheck** (bool enable, double limit)
	The method sets power sweep max output dc current condition.
bool	**SetPowerSweepMaxEvmCheck** (bool enable, double limit)
	The method sets power sweep max evm condition.
bool	**SetPowerSweepMaxPeakInCheck** (bool enable, double limit)
	The method sets power sweep max input peak power condition.
bool	**SetPowerSweepMaxPeakOutCheck** (bool enable, double limit)
	The method sets power sweep max output peak power condition.
bool	**SetPowerSweepMaxPInDelCheck** (bool enable, double limit)
	The method sets power sweep max PIn delivered condition.
bool	**SetPowerSweepMaxPOutCheck** (bool enable, double limit)
	The method sets power sweep max output power condition.
bool	**SetPowerSweepModParCompression** (bool enable, double limit)
	The method sets power sweep max PAR compression condition.
bool	**SetPulsedSettings** (double pulsePeriodUs, double pulseWidthUs, double measWindowUs, double measDelayUs, int average)
	The method configures the pulse settings of the RapidMS system when in Pulse mode.
bool	**SetReferenceLevel** (long referenceLevelId)
	The method sets a calibrated reference level to be used for measurement.
bool	**SetRfPulseMeasSettings** (double measWindowUs, double measDelayUs, int average)
	The method configures the rf pulse measure settings.
bool	**SetRfPulseSettings** (double pulsePeriodUs, double pulseWidthUs, double pulseDelayUs)
	The method configures the rf pulse settings.
bool	**SetRfSource** (string txSide, double power_dBm, double phase_deg)
	The method sets the rf source power and phase in degree.
bool	**SetSourceTargetPoint** (int harmonic, float targetReal, float targetImag)
	The method sets the source target point for impedance measurement.
bool	**SetSourceTargetPointPolar** (int harmonic, float targetMag, float targetPhaseDeg)
	The method sets the source target point for impedance measurement.
bool	**SetSparameterFrequencyList** (double[] frequencyList)
	The method sets the selected frequencies for s-parameter measurement.

bool **SetSweepTargetPoints** (bool isLoadSweep, float[] loadTargetReal, float[] loadTargetImag)	
	The method sets sweep target points to be measured during the impedance sweep.
bool **SetTargetFromS1P** (double[] frequencyListGHz, double[] sparMag, double[] sparPhaseDeg)	
	The method sets impedance targets from s1p format data.
bool **Spectate** ()	
	The method requests the RapidMS system to spectate system state.
bool **StartHardware** ()	
	The method starts and setup the hardware for measurements.
bool **StopHardware** ()	
	The method stops all hardware and closes any existing connections with them.
bool **TurnDCOnOff** (int dcSupply, bool turnOn)	
	The method turns the specified dc supply on/off.
bool **TurnDCOnOffAll** (bool turnOn)	
	The method turns all the configured dc supplies on/off.

Member Function Documentation

◆ AbortMeasurement()

```
bool AbortMeasurement ( )
```

The method aborts measurement if the system is running a sweep measurement.

Returns

True if the measurement is aborted successfully.

◆ Connect()

```
bool Connect ( string remoteMachineAddress,
               int    port,
               string key  )
```

The method lets remote user connect to the RapidMS system. The connect method must be called prior to any other method to confirm if the RapidMS system can be controlled via API. The RapidMS system ip address as well as the port at which the api is exposed must be passed to the method. If an API key is configured in the RapidMS software, then the key must also be provided as an argument for authentication.

Parameters

remoteMachineAddress	Ip address of the remote RapidMS system. Use localhost for same system.
port	Port for the connection endpoint (default port is 8733).
key	Key to authenticate the connection.

Returns

True if the connection request is successful.

◆ Disconnect()

```
bool Disconnect ( )
```

The method notifies the RapidMS system that the API connection needs to be disconnected.

Returns

True if the disconnection request is successful.

◆ DoMeasurement()

```
bool DoMeasurement ( string filePath )
```

The method requests the RapidMS system to perform measurement based on the applied settings. If it is a sweep measurement, then the measurement result is stored in the RapidMS system at the filePath locally. filePath is ignored if the system is setup for single measurement.

Parameters

filePath Target file path for sweep measurement. Ignored in single measurement.

Returns

True if the measurement started successfully.

◆ EnableTargetFromS1P()

```
bool EnableTargetFromS1P ( bool enableS1pTarget )
```

The method configures if the impedance targets should be set from s1p data.

Parameters

enableS1pTarget Enable/disable setting target from s1p data.

Returns

True if target from s1p flag is set correctly.

◆ GetAvailableWaveforms()

```
string[] GetAvailableWaveforms ( )
```

The method gets the list of modulated waveforms configured in the system. This method is supported only by the VT & DT system for now.

Returns

The list of modulated waveforms available in the RapidMS system.

◆ GetDCCurrent()

```
double GetDCCurrent ( int dcSupply )
```

The method gets the dc current measured on the specified dc supply.

Parameters

dcSupply DC supply: Input = 0, Output = 1, Aux1 = 2, Aux2 = 3

Returns

Current measured on the specified dc supply in mA.

◆ GetDCOnOffStatus()

```
bool GetDCOnOffStatus ( int dcSupply )
```

The method requests the status of the dc supply.

Parameters

dcSupply DC supply: Input = 0, Output = 1, Aux1 = 2, Aux2 = 3

Returns

True if the dc is turned on.

◆ GetDCVoltage()

```
double GetDCVoltage ( int dcSupply )
```

The method gets the dc voltage measured on the specified dc supply.

Parameters

dcSupply DC supply: Input = 0, Output = 1, Aux1 = 2, Aux2 = 3

Returns

Voltage measured on the specified dc supply in V.

◆ GetDeviceInfo()

```
string GetDeviceInfo ( )
```

The method returns the RapidMS system information.

Returns

Returns the connected RapidMS system information.

◆ GetInitParams()

```
string GetInitParams ( )
```

The method returns the initialization parameters representing the initial state of the system for the selected mode as a json string.

Returns

Initialization parameters as a json string.

◆ GetMeasModeStatus()

```
bool GetMeasModeStatus ( )
```

The method checks if the system is ready for measurement.

Returns

True if the system is in measurement mode.

◆ GetMeasuredABWave()

```
bool GetMeasuredABWave ( int          harmonic,
                         string       selWaveform,
                         out double[] waveform_real,
                         out double[] waveform_imag )
```

In modulated mode, the method returns last measured corrected time domain A / B waves for the specified harmonic in complex format.

Parameters

harmonic	Specify the harmonic for which A / B waves to be returned.
selWaveform	Specify which A / B wave to be returned. Valid values are A1, A2, B1 & B2.
waveform_real	Out argument for real part of measured corrected time domain A / B wave.
waveform_imag	Out argument for imaginary part of measured corrected time domain A / B wave.

Returns

True if waveform has been retrieved.

◆ GetPassiveTunerCal()

```
int[] GetPassiveTunerCal ( bool isLoad )
```

The method gets the passive tuner calibrations for selected frequency stored in the passive tuner connected to the system.

Parameters

isLoad Specify load or source side selected tuner.

Returns

Available tuner calibration Ids.

◆ GetReferenceLevels()

```
string GetReferenceLevels ( )
```

The method gets the list of calibrated reference levels as a json string containing array of reference level details.

Returns

Array of reference level details as a json string.

◆ GetSelectedFrequencies()

```
double[] GetSelectedFrequencies ( )
```

The method returns the frequencies selected for measurement.

Returns

Selected frequency list in GHz.

◆ GetSingleMeasuredData() [1/2]

```
string GetSingleMeasuredData ( )
```

The method returns the measurement result from single impedance measurement or s-parameter measurement as a json string.

Returns

Measurement result object as json string.

◆ GetSingleMeasuredData() [2/2]

```
bool GetSingleMeasuredData ( out string[] names ,
                             out double[] values )
```

The method returns the measurement result from single impedance measurement or s-parameter measurement as out parameters. The list of variable names and their values are returned separately.

Parameters

names List of available data from measurement

values List of respective measured/calculated values

Returns

True if the data is fetched successfully.

◆ GetStatus()

```
string GetStatus ( )
```

The method requests the status of RapidMS system.

Returns

Status of the system / which mode the system is in.

◆ GetSweepData()

```
string GetSweepData ( )
```

The method returns the measurement result from sweep measurement as a json string.

Returns

Measurement result as a string containing array of json string objects.

◆ IsMeasurementCompleted()

```
bool IsMeasurementCompleted ( )
```

The method checks if the measurement has completed or not.

Returns

True if measurement is completed or not running.

◆ LoadInitializationFile()

```
bool LoadInitializationFile ( string filePath )
```

The method sets the initialization file to be loaded for impedance measurements. The filePath must be the file path of the initialization file in the RapidMS system.

Parameters

filePath Path of the initialization file to be loaded in RapidMS system.

Returns

True if the initialization file is set successfully.

◆ LoadSourceCalibration()

```
bool LoadSourceCalibration ( string filePath )
```

The method lets the user load the specified source calibration file in the RapidMS software. The filePath is the file path of the source calibration file in the RapidMS system and not in the client system.

Parameters

filePath File path of the source calibration file in the RapidMS system.

Returns

True if the source calibration file is loaded successfully.

◆ LoadVectorCalibration()

```
bool LoadVectorCalibration ( string filePath )
```

The method lets remote user load the specified vector calibration file in the RapidMS software. The filePath is the file path of the vector calibration file in the RapidMS system and not in the client system.

Parameters

filePath File path of the vector calibration file in the RapidMS system.

Returns

True if the vector calibration file is loaded successfully.

◆ SetApullMode()

```
bool SetApullMode ( bool isApull )
```

The method sets A-pull mode instead of Load-pull mode.
* This feature is currently under testing and must not be used.

Parameters

isApull Specify if the system should be in A-pull mode or load pull mode.

Returns

True if the mode is set to A-pull successfully.

◆ SetCenterFrequency()

```
bool SetCenterFrequency ( double frequency )
```

The method sets the center frequency for impedance measurement. The target frequency must be in the calibrated list of frequencies.

Parameters

frequency Selected frequency in GHz.

Returns

True if the center frequency is set successfully.

◆ SetContinuousRf()

```
bool SetContinuousRf ( bool keepRfOn )
```

The method sets the flag to keep rf on continuously. By default, after single or sweep measurement, the rf is turned off. If the keep rf on continuously flag is set to true, then the rf is not turned off after the single or sweep measurement. In case of sweep measurement, the last rf power set will be kept unchanged.

Parameters

keepRfOn Specify if rf should be kept on/off after measurement.

Returns

True if the keep rf on continuously flag is set successfully.

◆ SetDCBias()

```
bool SetDCBias ( int    dcSupply,
                 double setVoltage_V )
```

The method sets the voltage of the specified dc supply.

Parameters

dcSupply DC supply: Input = 0, Output = 1, Aux1 = 2, Aux2 = 3

setVoltage_V Voltage to be set in V.

Returns

True if the dc voltage is set successfully.

◆ SetIFBandwidth()

```
bool SetIFBandwidth ( int targetBandwidthHz )
```

The method sets the IF bandwidth of the system when in CW mode.

Parameters

targetBandwidthHz IF frequency bandwidth in Hz.

Returns

True if the IF bandwidth is set correctly.

◆ SetInitialization()

```
bool SetInitialization ( double initPowerdBm,
                         string saveFilePath,
                         bool   skipGainCheck )
```

The method sets the initialization settings.

Parameters

initPowerdBm Initialization power in dBm.

saveFilePath File name including the path where the initialization file has to be saved.

skipGainCheck Skip gain error check. Must be set to false unless when debugging.

Returns

True if the initialization settings are set correctly.

◆ SetLoadSourcepull()

```
bool SetLoadSourcepull ( int    harmonic,
                         bool   enableLoadpull,
                         bool   enableSourcepull,
                         int    iterations,
                         double accuracydB )
```

The method sets impedance measurement settings for load-pull and source-pull.

Parameters

enableLoadpull	Enable load-pull for measurement.
enableSourcepull	Enable source-pull for measurement.
iterations	Maximum number of iterations.
accuracydB	Accuracy in dB.

Returns

True if the settings are set correctly.

◆ SetLoadTargetPoint()

```
bool SetLoadTargetPoint ( int   harmonic,
                          float targetReal,
                          float targetImag )
```

The method sets the load target point for impedance measurement. The target point is specified in real and imaginary format.

Parameters

harmonic	Harmonic for which the target to be set. The VT system supports only fundamental.
targetReal	Real part of load target point for the specified harmonic.
targetImag	Imaginary part of load target point for the specified harmonic.

Returns

True if the target point is set correctly.

◆ SetLoadTargetPointPolar()

```
bool SetLoadTargetPointPolar ( int   harmonic,
                               float targetMag,
                               float targetPhaseDeg )
```

The method sets the load target point for impedance measurement. The target point is specified in magnitude and phase format.

Parameters

harmonic	Harmonic for which the target to be set. The VT system supports only fundamental.
targetMag	Magnitude part of load target point for the specified harmonic.
targetPhaseDeg	Phase part of load target point for the specified harmonic in degree.

Returns

True if the target point is set correctly.

◆ SetMeasurementMode()

```
bool SetMeasurementMode ( int mode,
                          int measType )
```

The method sets the measurement mode and measurement category. This should be done when the software is in settings mode and before calling start hardware.

Parameters

mode	Measurement mode: CWMode = 0, PulsedMode = 1, ModulatedMode = 2
measType	Measurement category: Impedance = 0, SParameter = 1

Returns

True if the measurement mode and category are set correctly.

◆ SetMeasurementType()

```
bool SetMeasurementType ( bool isSweep )
```

The method sets the impedance measurement type (if it's single or sweep).

Parameters

isSweep True if sweep measurement is to be performed.

Returns

True if the measure type is set correctly.

◆ SetModulatedBandwidth()

```
bool SetModulatedBandwidth ( int targetBandwidth )
```

The method sets the modulated bandwidth requested by the user when the system is in Modulated mode. This method is supported only by the VT & DT system for now.

Parameters

targetBandwidth Supported bandwidth: BW50MHz = 50, BW100MHz = 100, BW250MHz = 250, BW500MHz = 500 and BW1000MHz = 1000

Returns

True if the bandwidth is set correctly.

◆ SetModulatedWaveform()

```
bool SetModulatedWaveform ( string waveformName )
```

The method sets the selected modulated waveform to be measured when in Modulated mode. The selected waveform must in the available list of waveforms in the software. The user can get the list of available waveforms using the GetAvailableWaveforms method. This method is supported only by the VT & DT system for now.

Parameters

waveformName Modulated waveform name to be selected.

Returns

True if the waveform is selected successfully.

◆ SetPassiveTunerCal()

```
bool SetPassiveTunerCal ( bool isLoad,
                          int  tunerCalId )
```

The method sets the selected passive tuner calibration when passive tuner is connected to the system.

Parameters

isLoad Specify load or source side selected tuner.

tunerCalId Tuner calibration Id to be set.

Returns

True if the target calibration is set successfully.

◆ SetPowerLevel()

```
bool SetPowerLevel ( double powerLeveldBm )
```

The method sets the power level for measurement in dBm.

Parameters

powerLeveldBm Target power in dBm.

Returns

True if the target power level is applied successfully.

◆ SetPowerSweep()

```
bool SetPowerSweep ( bool   enable,
                     double startdBm,
                     double stopdBm,
                     double stepdBm  )
```

The method enables and sets the power sweep settings. The power sweep if enabled is performed nested with impedance sweep. For the power sweep to be enabled, the impedance measurement must be set to sweep mode.

Parameters

enable Enable/disable power sweep measurement.

startdBm Start power level in dBm.

stopdBm Stop power level in dBm.

stepdBm Step in dBm.

Returns

True if the power sweep settings are applied correctly.

◆ SetPowerSweepCompression()

```
bool SetPowerSweepCompression ( bool   stopAtCompression,
                                bool   useMaxGain,
                                double compressiondBm  )
```

The method sets the power sweep compression condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

stopAtCompression Enable/disable stop at power level compression check.

useMaxGain Set the flag to check against max gain; otherwise, initial gain is used.

compressiondBm Compression level in dB to be validated against.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxAcprAdjCheck()

```
bool SetPowerSweepMaxAcprAdjCheck ( bool   enable,
                                    double limit  )
```

The method sets the power sweep max ACPR adjacent condition in modulated mode which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max ACPR adjacent check.

limit Max ACPR adjacent in dBc to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxDcInICheck()

```
bool SetPowerSweepMaxDcInICheck ( bool   enable,
                                  double limit  )
```

The method sets the power sweep max input dc current condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max input dc current check.

limit Max input dc current in mA to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxDcOutICheck()

```
bool SetPowerSweepMaxDcOutICheck ( bool   enable,
                                   double limit  )
```

The method sets the power sweep max output dc current condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max output dc current check.

limit Max output dc current in mA to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxEvmCheck()

```
bool SetPowerSweepMaxEvmCheck ( bool   enable,
                                double limit  )
```

The method sets the power sweep max evm condition in modulated mode which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max evm check.

limit Max evm in percentage to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxPeakInCheck()

```
bool SetPowerSweepMaxPeakInCheck ( bool   enable,
                                   double limit  )
```

The method sets the power sweep max input peak power condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max input peak power check.

limit Max input peak power in dBm to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxPeakOutCheck()

```
bool SetPowerSweepMaxPeakOutCheck ( bool   enable,
                                    double limit  )
```

The method sets the power sweep max output peak power condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max output peak power check.

limit Max output peak power in dBm to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxPInDelCheck()

```
bool SetPowerSweepMaxPInDelCheck ( bool   enable,
                                   double limit  )
```

The method sets the power sweep max PIn delivered condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max PIn delivered check.

limit Max PIn delivered in dBm to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepMaxPOutCheck()

```
bool SetPowerSweepMaxPOutCheck ( bool   enable,
                                 double limit  )
```

The method sets the power sweep max output power condition which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max output power check.

limit Max output power level in dBm to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPowerSweepModParCompression()

```
bool SetPowerSweepModParCompression ( bool   enable,
                                      double limit  )
```

The method sets the power sweep max PAR compression condition in modulated mode which will be validated during power sweep measurements. The power sweep for the current impedance point will be aborted and the system will move on to the next impedance point if the condition is met.

Parameters

enable Enable/disable stop at max PAR compression check.

limit Max PAR compression in dB to be validated against if enabled.

Returns

True if the settings are applied correctly.

◆ SetPulsedSettings()

```
bool SetPulsedSettings ( double pulsePeriodUs,
                         double pulseWidthUs,
                         double measWindowUs,
                         double measDelayUs,
                         int    average  )
```

The method configures the pulse settings of the RapidMS system when in Pulse mode.

Parameters

pulsePeriodUs Pulse period in µs.

pulseWidthUs Pulse width in µs.

measWindowUs Measurement window width in µs.

measDelayUs Measurement delay width in µs.

average No of averages to be measured.

Returns

True if the pulse settings are set correctly.

◆ SetReferenceLevel()

```
bool SetReferenceLevel ( long referenceLevelId )
```

The method sets a reference level from the list of calibrated reference level to be used for measurement. The reference level to be selected must be specified using an id from the calibrated reference level details.

Parameters

referenceLevelId Id corresponding to the reference level to be selected for measurement.

Returns

True if the specified reference level is set successfully.

◆ SetRfPulseMeasSettings()

```
bool SetRfPulseMeasSettings ( double measWindowUs,
                              double measDelayUs,
                              int    average )
```

The method configures the rf pulse measurement settings of the RapidMS system when in Pulse mode.

Parameters

measWindowUs Measurement window width in µs.

measDelayUs Measurement delay width in µs.

average No of averages to be measured.

Returns

True if the pulse measure settings are set correctly.

◆ SetRfPulseSettings()

```
bool SetRfPulseSettings ( double pulsePeriodUs ,
                          double pulseWidthUs ,
                          double pulseDelayUs  )
```

The method configures the rf pulse settings of the RapidMS system when in Pulse mode.

Parameters

pulsePeriodUs Pulse period in µs.

pulseWidthUs Pulse width in µs.

pulseDelayUs Pulse delay in µs.

Returns

True if the rf pulse settings are set correctly.

◆ SetRfSource()

```
bool SetRfSource ( string txSide ,
                   double power_dBm ,
                   double phase_deg  )
```

The method sets the rf source power and phase in degree.

Parameters

txSide Specify the rf source to be set (TxSource1F0, TxLoad1F0, TxSource2F0, TxLoad2F0,TxSource3F0, TxLoad3F0).

power_dBm Specify the rf power to be set to the source in dBm.

phase_deg Specify the phase to be set to the rf source in degrees.

Returns

True if the rf power and phase are set successfully.

◆ SetSourceTargetPoint()

```
bool SetSourceTargetPoint ( int   harmonic,
                            float targetReal,
                            float targetImag )
```

The method sets the source target point for impedance measurement. The target point is specified in real and imaginary format.

Parameters

harmonic Harmonic for which the target to be set. The VT system supports only fundamental.

targetReal Real part of source target point for the specified harmonic.

targetImag Imaginary part of source target point for the specified harmonic.

Returns

True if the target point is set correctly.

◆ SetSourceTargetPointPolar()

```
bool SetSourceTargetPointPolar ( int   harmonic,
                                 float targetMag,
                                 float targetPhaseDeg )
```

The method sets the source target point for impedance measurement. The target point is specified in magnitude and phase format.

Parameters

harmonic Harmonic for which the target to be set. The VT system supports only fundamental.

targetMag Magnitude part of source target point for the specified harmonic.

targetPhaseDeg Phase part of source target point for the specified harmonic in degree.

Returns

True if the target point is set correctly.

◆ SetSparameterFrequencyList()

```
bool SetSparameterFrequencyList ( double[] frequencyList )
```

The method sets the selected frequencies for s-parameter measurement. The selected frequencies must be in the calibrated list of frequencies.

Parameters

frequencyList Selected frequency list in GHz.

Returns

True if S-parameter frequency list is set successfully.

◆ SetSweepTargetPoints()

```
bool SetSweepTargetPoints ( bool    isLoadSweep,
                            float[] loadTargetReal,
                            float[] loadTargetImag )
```

The method sets sweep target points to be measured during the impedance sweep. The target points are specified in real and imaginary format.

Parameters

isLoadSweep Set true if load side needs to be swept.

loadTargetReal Set the real part of load or source sweep target points.

loadTargetImag Set the imaginary part of load or source sweep target points.

Returns

True if the target points are set correctly.

◆ SetTargetFromS1P()

```
bool SetTargetFromS1P ( double[] frequencyListGHz,
                        double[] sparMag,
                        double[] sparPhaseDeg )
```

The method sets impedance targets from s1p format data. The target points matching the center frequency are set. The target points are specified in magnitude and phase format.

Parameters

frequencyListGHz	Frequency target list of s1p data in GHz.
sparMag	Magnitude part of the corresponding target points.
sparPhaseDeg	Phase part of the corresponding target points.

Returns

True if the targets are set correctly.

◆ Spectate()

```
bool Spectate ( )
```

The method requests the RapidMS system to measure without updating/altering the signal generator part. This is mostly used when the keep continuous rf on flag is set and the user want to check the status at certain intervals.

Returns

True if the requested operation has started successfully.

◆ StartHardware()

```
bool StartHardware ( )
```

The method starts and setup the hardware for measurements. The system will be put into load-pull or s-parameter measurement mode.

Returns

True if the hardware is setup for measurement without any error.

StopHardware()

```
bool StopHardware ( )
```

The method stops all hardware and closes any existing connections with them. It sets the software back to settings mode.

Returns

True if hardware stopped without error.

TurnDCOnOff()

```
bool TurnDCOnOff ( int dcSupply,
                   bool turnOn )
```

The method turns the specified dc supply on/off.

Parameters

dcSupply DC supply: Input = 0, Output = 1, Aux1 = 2, Aux2 = 3

turnOn Turn the supply on/off.

Returns

True if the dc supply is set successfully.

TurnDCOnOffAll()

```
bool TurnDCOnOffAll ( bool turnOn )
```

The method turns all the configured dc supplies on/off. The order in which they are turned on/off is based on what the user has configured in the dc supply settings in RapidMS software.

Parameters

turnOn Turn all the supplies on/off.

Returns

True if the dc supplies are set successfully.

Generated by doxygen 1.16.1